To demonstrate your 16-bit ALU on the Digilent Nexys A7-100T FPGA board, you run into a practical hardware bottleneck: [1]

- **The Problem:** Your ALU requires 36 total input bits (16 bits for A, 16 bits for B, and 4 bits for Opcode). However, the Nexys A7 board only has **16 physical slide switches**. [1]
- **The Solution:** You must build a **Top-Level Wrapper Module** in Verilog. This wrapper acts as a controller. It lets you use the 5 onboard pushbuttons to choose what the 16 slide switches are currently controlling. [1, 2]

Step 1: The Top-Level Verilog Wrapper (nexys_alu_top.v)

This module connects the 16 physical switches (SW), 5 pushbuttons (BTNx), and 16 LEDs (LED) to your 16-bit ALU. [1, 3]

- Hold **BTNC (Center)**: The 16 switches set **Operand A**.
- Hold **BTNU (Up)**: The 16 switches set **Operand B**.
- Release all buttons: The 16 switches automatically control the **Opcode** (using just SW[3:0]).
- The 16 physical LEDs continuously display the ALU output. [1]

```
module nexys_alu_top (
    input  [15:0] SW,        // 16 physical slide switches
    input  BTNC,             // Center button -> Sets Operand A
    input  BTNU,             // Up button     -> Sets Operand B
    output [15:0] LED,       // 16 physical LEDs for output
    output LED16_B,          // RGB LED Blue  -> Zero Flag
    output LED17_R           // RGB LED Red   -> Carry Flag
);

    // Internal registers to lock in the 16-bit values
    reg [15:0] reg_A = 16'b0;
    reg [15:0] reg_B = 16'b0;
    reg [3:0]  reg_Opcode = 4'b0;

    wire [15:0] alu_result;
    wire carry_flag;
    wire zero_flag;

    // Control how switches route data based on pushbuttons
    always @(*) begin
        if (BTNC) begin
            reg_A = SW;       // Lock current switch layout into
Input A
```

```
        end else if (BTNU) begin
            reg_B = SW;       // Lock current switch layout into
Input B
        end else begin
            reg_Opcode = SW[3:0]; // Use the first 4 switches as
the operation code
        end
    end

    // Instantiate your 16-bit ALU module
    alu_16bit my_alu (
        .A(reg_A),
        .B(reg_B),
        .Opcode(reg_Opcode),
        .ALU_Out(alu_result),
        .CarryOut(carry_flag),
        .Zero(zero_flag)
    );

    // Drive the board outputs
    assign LED     = alu_result;    // Display 16-bit calculation
output on the LEDs
    assign LED17_R = carry_flag;    // Light up Red RGB LED if math
overflows
    assign LED16_B = zero_flag;     // Light up Blue RGB LED if
output is exactly 0
endmodule
```

Step 2: The Xilinx Constraint File (nexys_constraints.xdc)

You need to tell Xilinx Vivado exactly which physical pins on the Artix-7 chip match your Verilog variables. Create a target .xdc file and paste the official Digilent Nexys A7-100T Master Pins text inside: [4, 5, 6, 7]

Create an .xdc file using the official Digilent Master Constraints file to map your module to the FPGA pins. Key assignments include: [4, 8]

- **Switches (SW[0-15]):** Pins J15, L16, M13, R15, R17, T18, U18, R13, T8, U8, R16, T13, H6, U12, U11, V10.
- **LEDs (LED[0-15]):** Pins H17, K15, J13, N14, R18, V17, U17, U16, V16, T15, U14, T16, V15, V14, V11, V12.
- **Buttons:** N17 (Center), M18 (Up). [4]

Map LED16_B (Blue) and LED17_R (Red) to the RGB LED pins on the board for flag status. [9]

Step 3: How to Run the Demonstration on Hardware

1. **Synthesize and Generate Bitstream:** In Vivado, implement your design and generate the bitstream. [10, 11, 12, 13, 14]
2. **Program the Board:** Use the Hardware Manager to connect to the Nexys A7 and program it via micro-USB. [15, 16]
3. **Operation Flow (e.g., $2 + 3 = 5$):**
    - Set switches to 0000_0000_0000_0010 (binary for 2). Hold BTNC to lock this into operand A.
    - Set switches to 0000_0000_0000_0011 (binary for 3). Hold BTNU to lock this into operand B.
    - Set switches SW[3:0] for your ALU operation (e.g., 0000 for addition).
    - Observe the 16 LEDs; they should display the binary result (...0101 for 5). [1]

Would you like to learn how to route this calculation output to the **7-Segment Display** digits on the board so you can read actual hexadecimal numbers instead of counting binary LEDs? [11]

[1] https://digilent.com
[2] https://www.instructables.com
[3] https://medium.com
[4] https://github.com
[5] https://www.youtube.com
[6] https://medium.com
[7] https://forum.digilent.com
[8] https://pcbsync.com
[9] https://svenssonjoel.github.io
[10] https://www.youtube.com
[11] https://www.youtube.com
[12] https://projectf.io
[13] https://xilinx.github.io
[14] https://community.element14.com
[15] https://github.com
[16] https://digilent.com